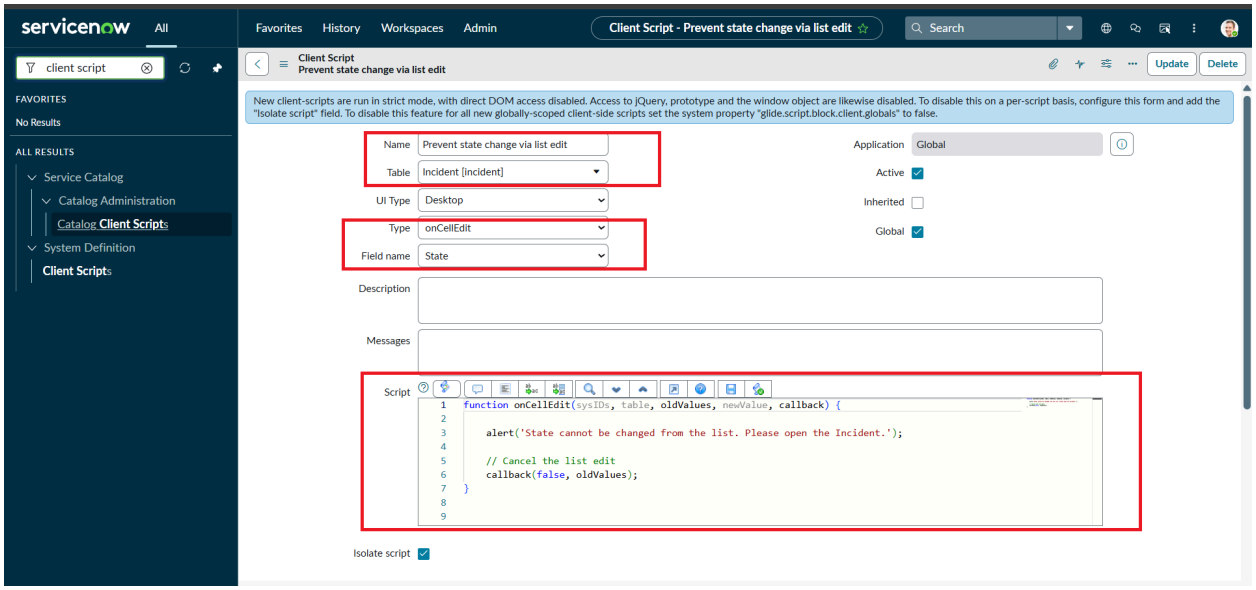


Task 6: Testing the Configuration

Activity 1: Test Mandatory Enforcement

1. **Navigate to Incident:**
   Go to Incident → Create New to open a new Incident form.

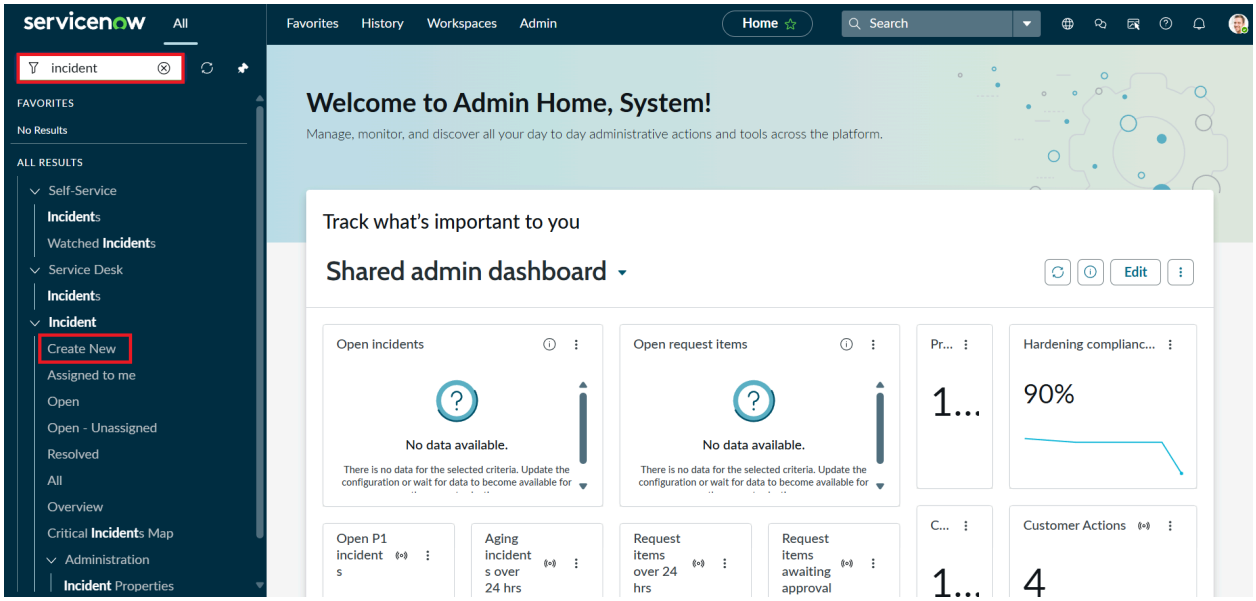
2. **Set the Impact Field:**
   Select **High** for the **Impact** field. This will trigger the UI Policy and Client Script behaviors you configured earlier.

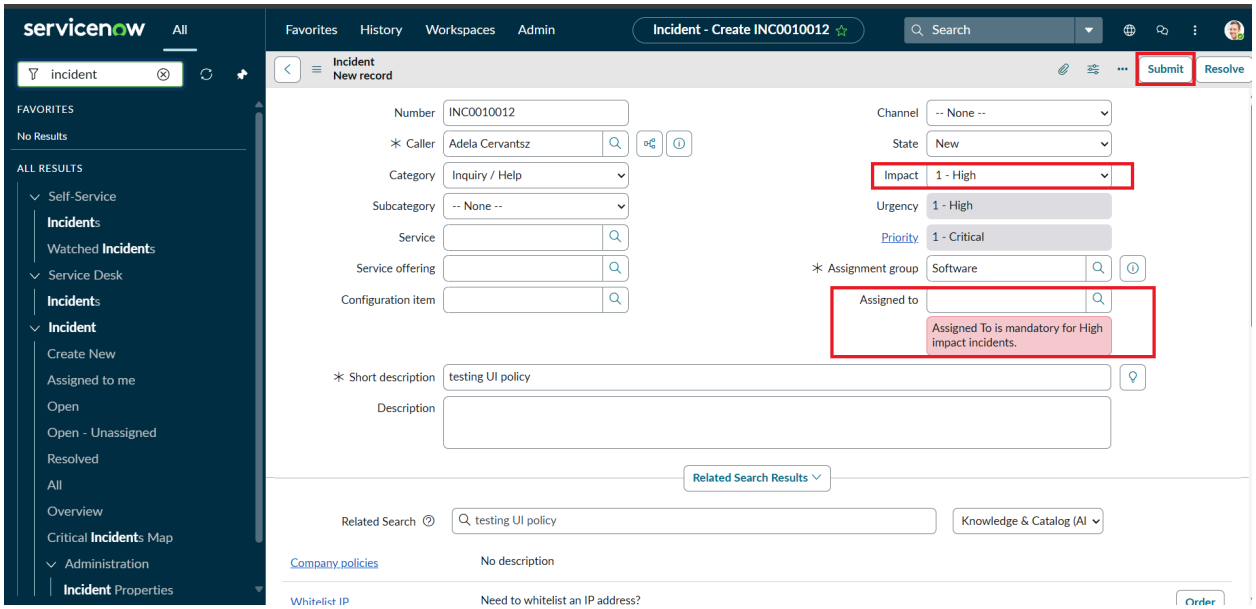
3. **Leave Assigned To Empty:**
   Do not fill in the **Assigned To** field. This will test the onSubmit Client Script that prevents saving if Assigned To is missing.

4. **Click Submit:**
   Attempt to save the Incident record.

5. **Verify the Result:**

- The Incident should **not be saved** because Assigned To is required when Impact is High.
- An **error message** should appear, indicating that the Assigned To field cannot be empty.
- Confirm that all UI Policy actions, such as read-only fields or automated urgency updates, are working as expected.





Activity 2: Test Successful Save

1. **Fill the Assigned To Field:**
   Open the Incident form and select a user in the **Assigned To** field. This ensures the onSubmit Client Script condition is satisfied.

2. **Click Submit:**
   Attempt to save the Incident record.

3. **Verify the Result:**

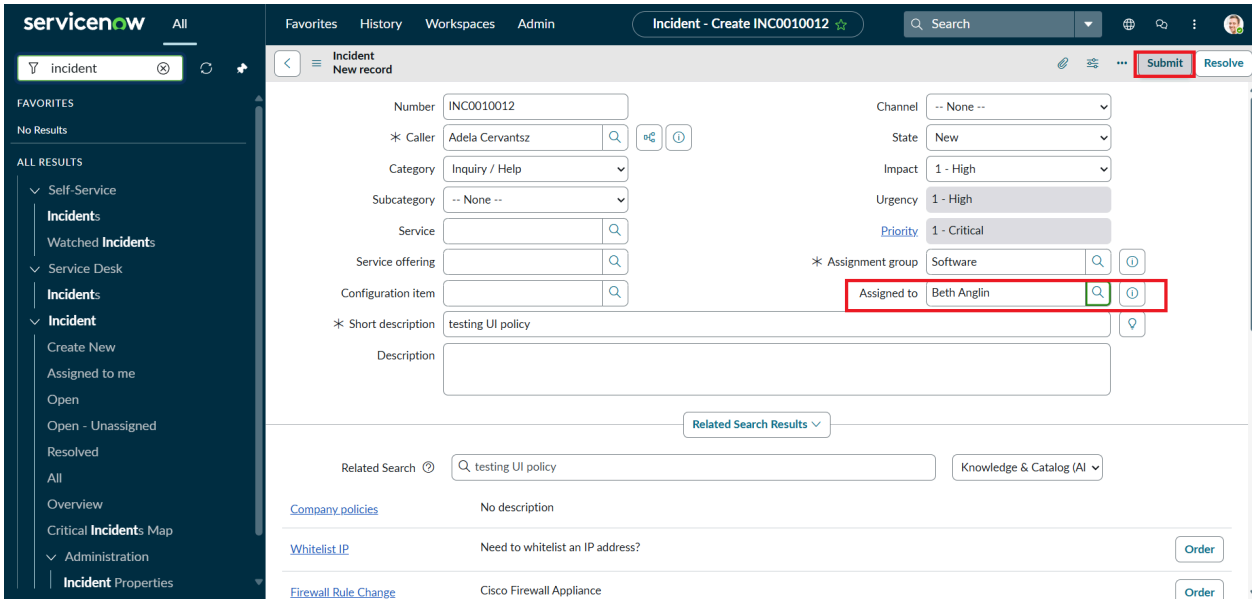
   - The Incident should be **saved successfully** without any errors.

   - Confirm that all other configured behaviors, such as:

     - **Urgency auto-setting** (from the onChange Client Script)

     - **Read-only fields** (from the UI Policy Action)

     - **State restrictions** (from the onCellEdit Client Script)
       are functioning as expected



Activity 3: Reverse Condition Test

1. **Open an Existing Incident:**
   Go to Incident → Open and select an Incident where **Impact** is currently set to **High**.

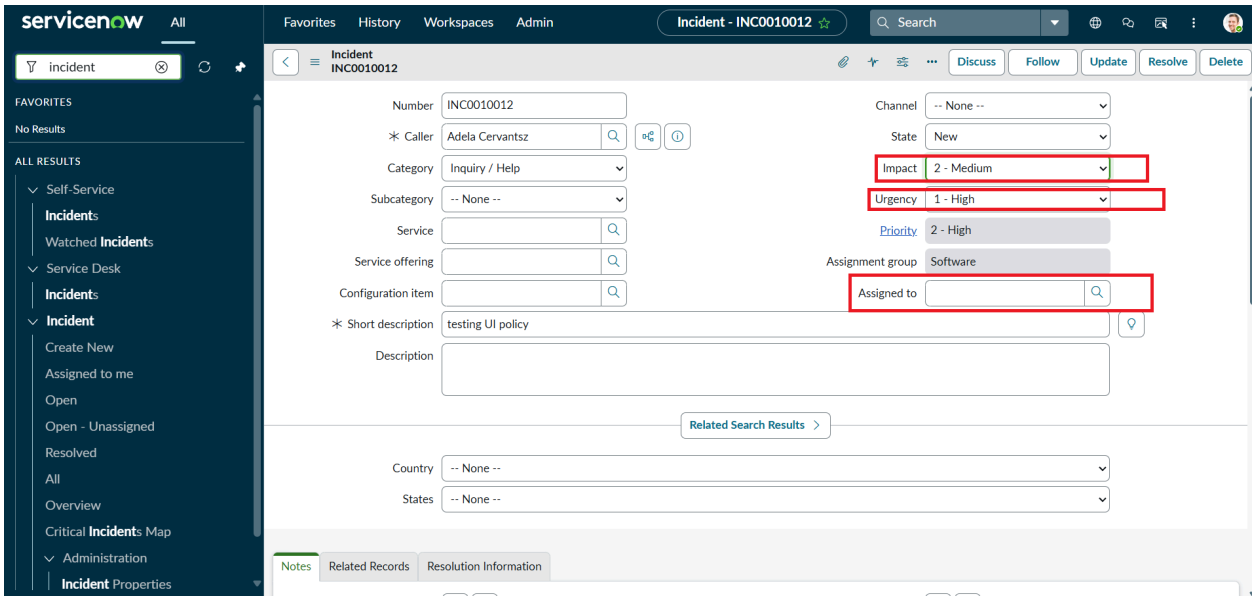
2. **Change Impact:**
   Update the **Impact** field from **High** to **Medium**. This will test the **Reverse if false** functionality of your UI Policy and the behavior of your Client Scripts.

3. **Verify the Field Behavior:**

   - **Assigned To** should no longer be mandatory.

   - **Urgency** should now be editable, meaning the read-only UI Policy action is reversed.

4. **Save the Record:**
   Click **Submit** to save the changes. The record should save successfully without errors, confirming that the reverse condition works as intended.



Activity 4: Test List Edit Blocking

1. **Navigate to the Incident List:**
   Go to Incident → All to view all Incident records in a list view.

2. **Attempt a List Edit:**

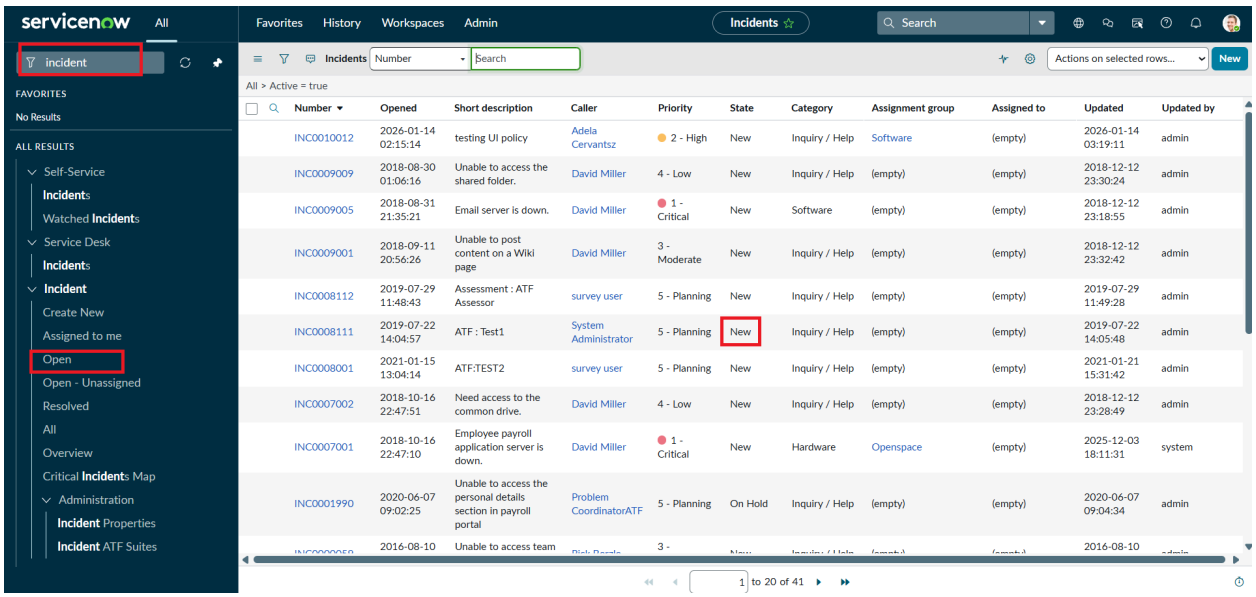
   - Locate any Incident in the list.

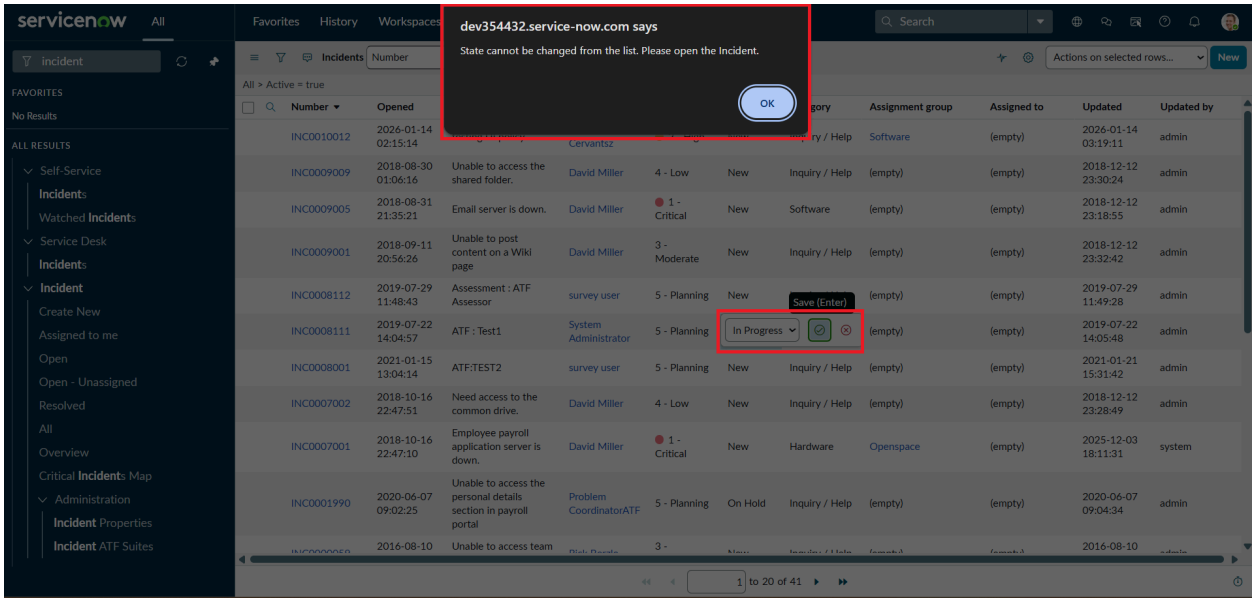
   - **Double-click the State field** to try editing it directly from the list.

3. **Verify the Behavior:**

   - An **alert message** should appear, indicating that direct changes to the State field are not allowed.

   - The **State value should remain unchanged**, confirming that the onCellEdit Client Script is working correctly.





Activity 5: Test Form-Based Update

1. **Open the Incident Record:**
   Navigate to the same Incident record you used in previous tests.

2. **Change the State:**
   On the **Incident form**, update the **State** field to a new value. This tests that changes via the form (not the list) are allowed.

3. **Click Update:**
   Save the record by clicking **Update**.

4. **Verify the Result:**

   - The **State change should be saved successfully**.

   - Confirm that other fields, such as **Assigned To** and **Urgency**, still follow the configured UI Policy and Client Script rules.